Computing Methods for Physics – 11 September 2023

Your exam material (code files, plots, datafiles, etc.) must be submitted via google classroom by 13:00 as a single `zip` file.

C++ evaluation will be based on: correct syntax, proper return types, proper arguments of functions, data members and class interfaces, unnecessary void functions, correct mathematical expressions, comments throughout the code, separation of class implementations and interfaces.

Python evaluation will be based on: correct syntax, avoiding C-style loops, using Python features in general, comments throughout the notebook/scripts, labels, legends and plot styling and clarity in general.

Part 1 – C++

Implement a class `PointList` to store and print to screen the 2D Cartesian coordinates of N points.

- It must include a constructor that takes N as an argument and then asks the user to input the coordinates from keyboard.
- It must have a method to swap points i and j.

Implement a class `Polygon` that inherits from `PointList` and represents a polygon by the coordinates of its vertices. `Polygon` must have methods to:

- print the coordinates of the vertices to screen, and the number of vertices;
- calculate the perimeter of the polygon (assume the user provides the vertices in order);
- swap two vertices;
- update the coordinates of the i-th vertex;
- print to screen the distance between the i-th vertex and all the other vertices;
- calculate the area of the smallest rectangle with sides parallel to x and y that contains the whole polygon.

Your submitted material must include a file `app.cpp` that showcases the classes you implemented.

Part 2 – Python

Build a class to handle the following system of first-order nonlinear differential equations

$$\begin{cases} \dfrac{dP}{dt} = \beta - (\zeta Z + \mu)P \\ \dfrac{dZ}{dt} = (\zeta - \delta)ZP + \rho D \\ \dfrac{dD}{dt} = (\mu + \delta Z)P - \rho D \end{cases},$$

where $t \geq 0$ is the independent variable, P, Z, and D depend upon t, and $\beta, \mu, \rho, \zeta, \delta \in \mathbb{R}^+$.

Use a Python notebook or Python scripts to complete the following tasks.

1. Provide the ability to initialize instances of the class by passing the input parameters $\beta, \mu, \rho, \zeta, \delta$ and the initial conditions $P(t = 0)$, $Z(t = 0)$, $D(t = 0)$.

2. Provide a method to simulate the evolution of $P(t)$, $Z(t)$, and $D(t)$ for a time T with a sampling time Δt. The resulting $\{t_i, P(t_i), Z(t_i), D(t_i)\}$ values must be saved to file and the integration must be handled with `odeint`.

3. Provide the ability to generate the following 5 graphs to display the results of a simulation: $\{P, Z, D\}$ versus t, $\{\frac{dP}{dt}, \frac{dZ}{dt}, \frac{dD}{dt}\}$ versus t, $\frac{dP}{dt}$ versus P, $\frac{dZ}{dt}$ versus Z, $\frac{dD}{dt}$ versus D.

4. Provide the ability to chose the integration method (with `odeint` as default). As an additional integration method, implement the Euler method. Given the Cauchy problem $du/dt = f(t, u)$, $u(t_0) = u_0$, and an integration step size Δt, the Euler method approximates the solution $u = u(t)$ with the discrete values

$$t_{i+1} = t_i + \Delta t \tag{1a}$$
$$u_{i+1} = u_i + f(t_i, u_i)\Delta t\,. \tag{1b}$$

   where $i = 0, \ldots, N$. The T introduced earlier is related to the parameters Δt and N by $T = N\Delta t$.

5. Choose a set of input parameters and compare the solutions provided by the integration method you implemented and `odeint`.